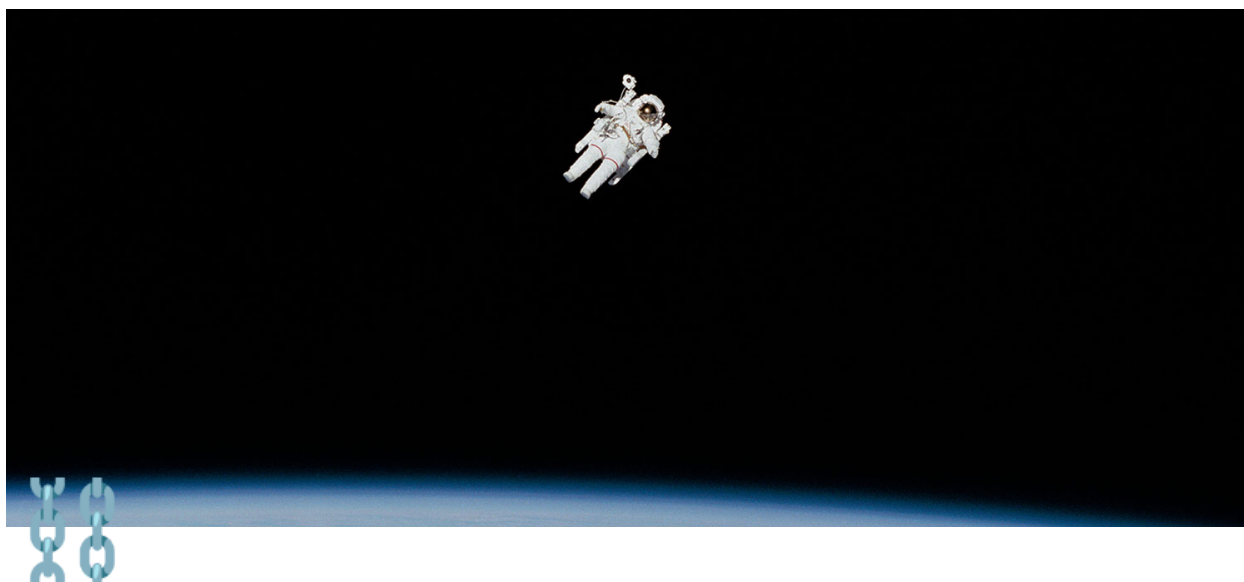




Компоненты CI/CD пайплайны и их взаимосвязь

Непрерывная интеграция и непрерывная доставка (CI/CD) — это подход к разработке программного обеспечения, в котором все разработчики работают вместе в общем репозитории кода и по мере внесения изменений существуют автоматизированные процессы сборки для обнаружения проблем с кодом. Результатом является более быстрый жизненный цикл разработки и более низкая частота ошибок.

Автоматизированные процессы сборки и обнаружения проблем с кодом (пайплайны, конвейеры) представляют собой набор действий (алгоритм) выполняемый над исходным кодом проекта.

Арсенал средств для CI/CD пайплайнов

В CI/CD пайплайнах применяются различные средства в зависимости от потребностей отдельной группы разработчиков, ограничений или условий среды и стеков разработки. К наиболее часто применяемым компонентам относятся:

1. *Распределенные системы контроля версий (РСКВ). Примеры - Git, Subversion.*

Слова излишни.

2. Средства контейнеризации (Docker, Podman).

Зачем нужно? Позволяет “запечатать” всю среду, необходимую для сборки и запуска приложения (по факту - создать снапшоты файловой системы) в т.н. “образы” для дальнейшего переиспользования без необходимости контроля среды на различных устройствах, ОС, ВМ и т.д. а также изолировать исполняемое приложение от других процессов в ОС для обеспечения безопасности

Как работает? На этапе сборки образа происходит выполнение заданных разработчиком команд (в случае с Docker, они указываются в файле `Dockerfile`), результаты преобразования этими командами файлов в ФС сохраняются в виде снапшотов ФС и затем сохраняются в образ, хранящий все необходимые для работы приложения файлы

Как пользоваться? В случае с Docker, комплект поставки включает в себя два компонента - клиент и сервер. Сервер работает как демон в Linux, получая команды для работы на открытый socket (`/var/run/docker.sock`). Команды могут быть отправлены как собственным клиентом Docker, так и любым другим приложением, способным подключаться к API Docker.

Типовые команды для работы с Docker:

- `docker build` - сборка образа из Dockerfile
- `docker images` - список собранных образов
- `docker run` - запуск контейнера из образа
- `docker ps` - список контейнеров
- `docker stop` - остановка контейнера
- `docker logs` - подключиться к выводу STDOUT и STDERR контейнера Docker
- `docker exec` - выполнить команду внутри работающего контейнера
- `docker attach` - STDIN, STDOUT и STDERR контейнера Docker

3. Модель ветвления распределенных систем контроля версий (GitFlow, Trunk, GitHub flow).

Зачем нужно? Позволяет формализовать работу с изменениями в коде, создать определенные правила их внесения и интеграции в кодовую базу приложения удобно и безопасно для всей команды разработчиков.

Как работает? В начале работы над проектом создается набор правил, в частности, того, как требуется оформлять новые доработки проекта (исправление ошибок, доработку функционала), как организовывается иерархия веток, в каком порядке, при каких условиях и из какой в какую ветку происходят сливания. Также организовывается работа по ревью кода - проверками одним или несколькими разработчиками законченного кода своего коллеги, готовящегося к вливанию в другую ветку.

Как пользоваться? Для удобства стоит выделить хотя бы две ветки - ветку релизную (последний коммит в ней всегда должен быть на проде) и ветку “в разработке” (последний коммит в ней всегда должен быть на тестовом стенде). В зависимости от политики тестирования ПО, количества тестовых стендов и различных особенностей проекта можно выделить также и дополнительные ветки. Также следует определить порядок именования веток в которых происходит создание нового кода и правка старого а также политику их вливания в другие ветки (общим перманентным правилом является то, что нерабочий (некомпилируемый, крашающийся на старте) код никогда не коммитится во избежание аварий).

4. Сервера непрерывной интеграции и развертывания (*Jenkins, Drone IO, TeamCity*)

Зачем нужно? Предоставляют возможность конфигурации и выполнения шагов пайплайна.

Как работает? В общем виде сервер непрерывной интеграции подключается разработчиком к репозиторию разрабатываемого проекта, после чего начинает отслеживать изменения в проекте и, после конфигурации шагов и условий их запуска, начинает выполнять описанные шаги при помощи “агентов” - программ, которые получают копию исходного кода приложения и выполняют с ним действия, описанные в шагах пайплайна (сборку, тестирование, развертку).

Как пользоваться? Необходимо развернуть минимум две составляющие - сервер и агент, подключить их к своему репозиторию, настроить команды, необходимые для сборки проекта и также настроить правила запуска отдельных или групп шагов.

5. Статические анализаторы (*Bandit, Trufflehog, Gosec*)

Зачем нужно? Проводят предварительные проверки исходного кода по определенным правилам - правилам стиля кода (Code Quality, Code Style

анализаторы), правилам безопасности кода (Static Application Security Testing) а также проводят композиционный анализ (Software Compositional Analysis) для выявления используемых в программе сторонних компонентов (библиотек, пакетов) и их уязвимостей. Позволяют проверить код по разным параметрам до его сборки и использования.

Как работает? На вход анализатору подается исходный код проекта, далее, в зависимости от используемого анализатора, проводятся проверки на наличие вхождений в коде каких-либо языковых (синтаксических) конструкций, которые являются небезопасными/устаревшими/нежелательными. Статический анализ безопасности также может включать в себя поиск “секретов” то есть хранящихся статически в коде паролей или токенов, а также графа выполнения кода с просчетом всех возможных значений переменных и функций для проверки того, что код не обладает недеklarированными возможностями.

Как пользоваться? Зачастую анализаторы поставляются в виде бинарных файлов либо собранных docker-образов. Результатом работы обоих видов поставки могут быть:

- Текстовые файлы с отчетами
- Вывод в STDOUT/STDERR
- Завершение работы с кодом отличным от 0 для сигнализации обнаруженных проблем
- (В редких случаях) Отправка информации о проблемах в виде HTTP запросов на CI/CD сервер либо удаленный репозиторий.

Для использования таких анализаторов в CI/CD пайплайне необходимо добавить шаг, вызывающий анализатор и считывающий результаты его работы. При обнаружении критичных проблем сборку стоит прекратить (в целях экономии ресурсов, так как проблемный код использовать опасно и результаты сборки использованы не будут).

6. Автотесты

Зачем нужно? Проверить, что код работает так, как ожидалось, после того, как он был собран.

Как работает? Заранее написанные функции на языке разрабатываемого приложения (зачастую, с использование специализированной библиотеки либо

фреймворка для тестирования), подключаются к коду и, выполняя заранее прописанные разработчиком действия, определяют, правильно ли код выполняет свои функции. Существуют различные виды тестов, включая юнит-тесты (проверяют отдельные части кода обособленно) и интеграционные тесты (проверяют несколько составляющих кода при их совместной работе).

Как пользоваться? В соответствии с используемым стеком, для каждой из существующих функций (методов) в коде разработать по одной функции, работа которой будет заключаться в проверке корректности выполнения функции при определенных заранее описанных параметрах. После сборки проекта проводить тестирование каждой функции на предмет того, не сломало ли ее предыдущее обновление в коде.

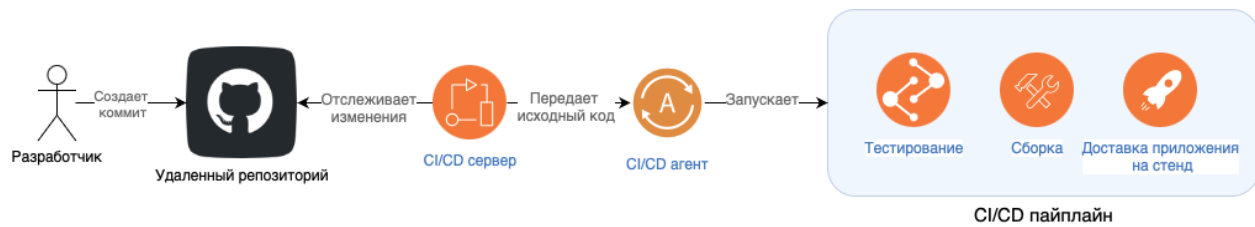
7. Средства миграции БД (*Liquibase* и другие)

Зачем нужно? Синхронизировать схемы (и иногда содержимое) баз данных приложения на различных стендах, к примеру, при добавлении нового поля или новой таблицы, либо удаления не нужных индексов.

Как работает? Детали работы различных средств уникальны в каждом случае, но в целом, такие системы осуществляют подключение к СУБД на исследуемых стендах, с помощью встроенного функционала СУБД исследуют содержимое различных баз данных, таблиц, их индексов и других ограничений, а также, в редких случаях, данных и, в зависимости от выбранного (либо predeterminedного разработчиками средства) способа вывода осуществляют либо автоматизированное выполнение команд SQL для приведения двух БД к одному виду, либо предоставляют разработчику скрипт, который совершает эти действия (для проверки скрипта самим разработчиком). В случае неудачи, такие системы, как правило, оснащены механизмом “отката” для возврата к начальному состоянию БД.

Как пользоваться? Перед выкаткой новой версии приложения на следующий стенд проверять, существуют ли изменения в схемах базы данных и, если они есть, запускать скрипт по их приведению к общему виду.

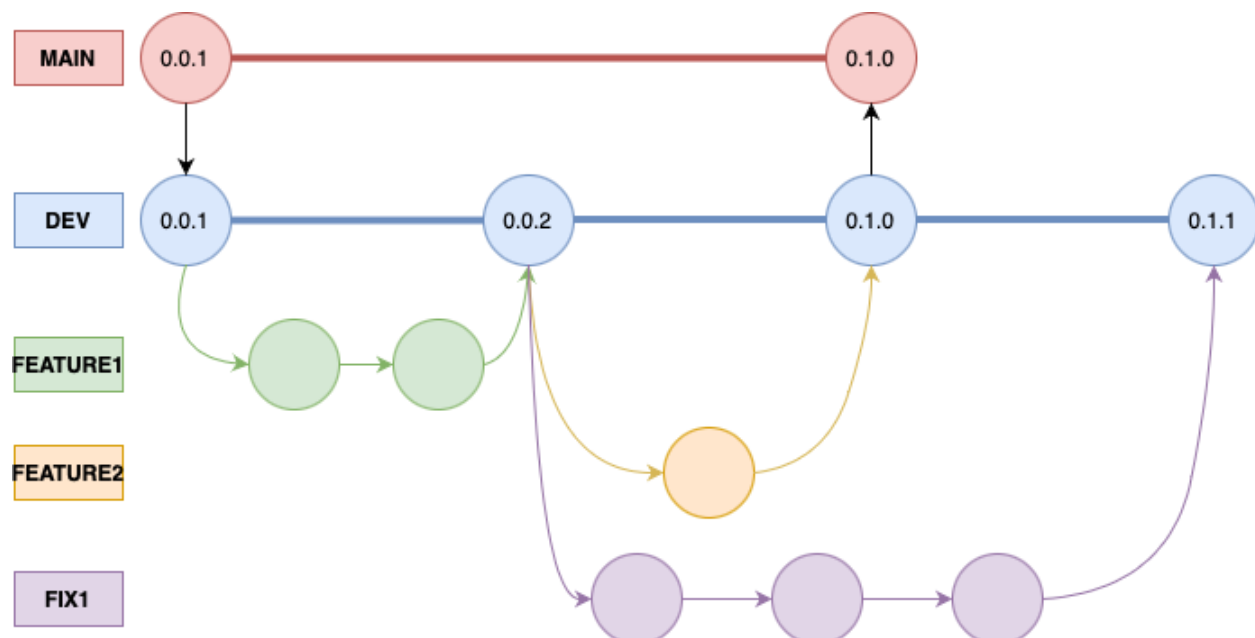
Типовой состав и последовательность шагов в CI/CD пайплайне



Основные “действующие лица” CI/CD пайплайна и их действия.

Рассмотрим на примере небольшого веб-приложения как можно организовать автоматизированное тестирование, сборку и доставку собранного приложения сначала на тестовый стенд (для проверки тестировщиками) а затем на продуктивный стенд - для клиентов.

В первую очередь, необходимо выбрать модель ветвления РСКВ. В качестве РСКВ будет использоваться Git. В качестве модели будет использоваться GitFlow, типовой порядок ветвления представлен ниже.



Политика создания и вливания веток следующая:

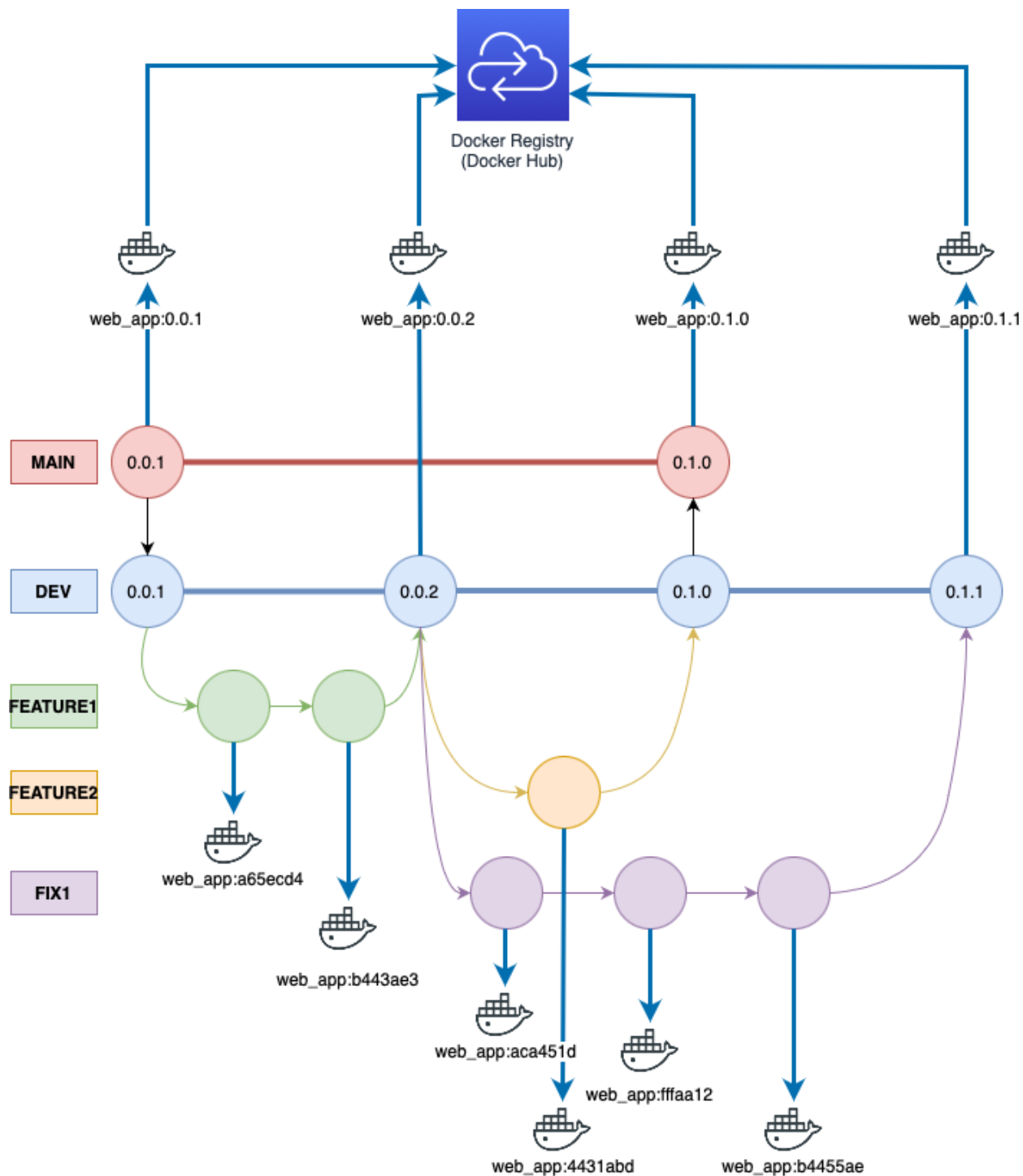
1. Ветка **main** - ветка релизов, последний коммит в этой ветке должен быть на prod-стенде. Коммиты в ветке появляются только за счет вливания в нее ветки

`dev`. Сливание ветки `dev` в ветку `main` осуществляется только после тестирования кода, залитого в ветку `dev`.

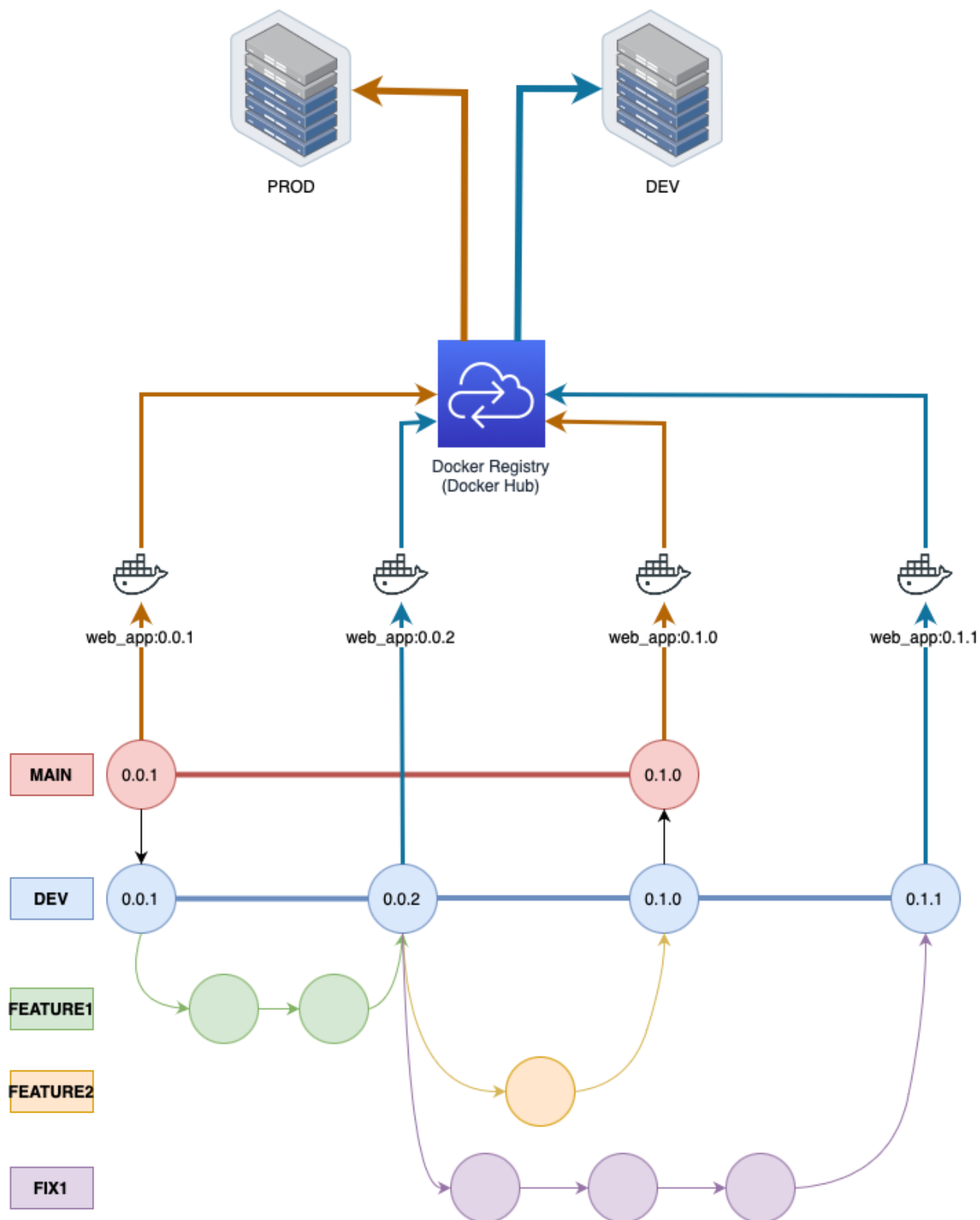
2. Ветка `dev` - ветка тестирующегося кода, последний коммит в этой ветке должен быть на stage-стенде. Коммиты в ветке появляются только за счет вливания в нее веток `FIX*` и `FEATURE*`. Сливание веток в ветку `dev` осуществляется только после код-ревью старшим коллегой-разработчиком.
3. Ветки `FEATURE*` и `FIX*` - ветки, коммиты в которые вносятся непосредственно разработчиками в рамках выполнения задач по разработке и доработке проекта. Последние коммиты в данных ветках расположены на машинах разработчиков, где и проходят первичную обкатку.

В качестве средства контейнеризации используется Docker. В ходе выполнения пайплайна происходит создание докер-образа с работающим скомпилированным приложением независимо от того, сборка какой ветки происходит. При сборке веток `dev` и `main`, в связи с необходимостью доставки образа на стенды, происходит отправка собранных образов в хранилище образов Docker - Docker Registry, в качестве которого выступает Docker Hub.

Связь коммитов, веток и образов приведена ниже.



После выгрузки собранных образов в облако, происходит их развертывание на соответствующих им стендах - образы, собранные из кода в ветке `dev` развертываются на DEV-стенде, а образы, собранные из кода в ветке `main` развертываются на PROD-стенде.



После развертывания и проверки успешной работы на PROD-стенде, команда продолжает доработки в соответствии с принятой моделью.